

Projeto Motiva - Checkpoint 2

Atualização remota de firmware (OTA) em ESP32

Ciência da Computação - 2º ano, 4º semestre

Professor Marcelo Fernando Morgantini

Integrante	RM
Lucas Kenzo Nishiwaki	561325
Felipe Hui Hattori	565169
Kauã Peres de Macedo	563977
Rafael Vaz de Lima	566429
André Eduardo Martins	563297
Kauany Ribeiro de Moura	564576

1. Objetivo

Desenvolvemos duas versões de um programa para simular o monitoramento da altura da vegetação. O ESP32 gera cinco leituras por sessão e pode receber uma nova versão pela internet, sem acesso físico ao equipamento. A versão 2.0 acrescenta mediana e histerese ao cálculo da média.

2. Projetos publicados

Repositório público: <https://github.com/luc916/Motiva-CP2-OTA>

Projeto público Wokwi: <https://wokwi.com/projects/475988461020262401>

Teste controlado de histerese: <https://wokwi.com/projects/475988911733956609>

Teste de erros: <https://wokwi.com/projects/475989558168554497>

Demonstração executada em 23/09/2026: o ESP32 iniciou em 1.0, consultou o GitHub após três sessões, gravou o firmware e reiniciou automaticamente em 2.0. A correção de reconexão Wi-Fi foi incluída e a OTA repetida com o binário final. Histerese e cinco cenários de erro também foram verificados no Wokwi.

O projeto principal usa dados pseudoaleatórios e sessões de 48 segundos. O projeto separado de histerese usa dados fixos e sessões de 12 segundos para reproduzir os limites. Os registros de execução estão nos anexos.

3. Arquitetura

ESP32 com FW 1.0 → Wokwi-GUEST → manifesto remoto → download do .bin → partição OTA → reinício → FW 2.0.

O circuito usa apenas ESP32 DevKit v1, LED RGB de cátodo comum e três resistores de 220 ohms. Não há sensor: as alturas são simuladas no programa.

Funcionamento do programa

4. Leituras e temporização

Cada sessão armazena cinco números inteiros pseudoaleatórios entre 10 e 20 cm. A chamada `random(10, 21)` inclui o 10 e exclui o 21. O programa mostra cada leitura e calcula a média usando a soma dividida por 5.0.

Evento	Instantes em relação à primeira sessão
Sessão 1	0, 2, 4, 6 e 8 segundos
Sessão 2	48, 50, 52, 54 e 56 segundos
Sessão 3	96, 98, 100, 102 e 104 segundos
Primeira consulta OTA	Após concluir a terceira sessão, perto de 104 s

O controle usa `millis()`. O próximo início é obtido somando 48000 ms ao início anterior. Assim, não se soma uma espera de 48 segundos ao fim das leituras. O laço verifica os prazos continuamente, com uma pequena pausa de 1 ms; pode haver pequena variação de execução, mas o período programado não acumula os 8 segundos de coleta.

5. Evolução da versão 2.0

A versão 1.0 mantém o LED azul. A versão 2.0 conserva as leituras e a média, copia o vetor e ordena essa cópia pelo método bubble sort. O vetor original permanece intacto. Para cinco valores, a mediana é o terceiro valor ordenado, no índice 2.

Exemplo de cálculo, não log da simulação: para 18, 12, 15, 14 e 20, a ordem crescente é 12, 14, 15, 18 e 20. A média é 15,8 cm e a mediana é 15 cm.

Mediana	Estado e indicação
Maior ou igual a 16 cm	ALERTA, LED vermelho
Entre 14 e 16 cm	Conserva o estado anterior e sua cor
Menor ou igual a 14 cm	NORMAL, LED verde

O estado inicial da versão 2.0 é NORMAL. Como as leituras são inteiras, a faixa intermediária é representada por 15 cm. Na sequência de medianas 15, 16, 15 e 14, os estados são NORMAL, ALERTA, ALERTA e NORMAL. A regra segue a tabela de condições do enunciado.

6. Ligações

GPIO 25 → resistor → R; GPIO 26 → resistor → G; GPIO 27 → resistor → B. Cada resistor tem 220 ohms. O terminal comum do LED vai ao GND. O arquivo `diagram.json` contém essas ligações.

Atualização e execução

7. Processo OTA

Depois de três sessões completas, uma tarefa de rede conecta o ESP32 à Wokwi-GUEST e consulta `version.json`. A comparação separa os números maior e menor da versão. Uma versão igual ou inferior não gera atualização. Quando existe uma versão superior, `HTTPUpdate` baixa a aplicação, grava a partição OTA inativa e, em caso de sucesso, o programa reinicia o ESP32.

A tarefa de rede mantém as esperas de Wi-Fi e HTTP fora do loop de medição. Se a tentativa falhar, o firmware atual continua e o programa tenta novamente depois de outra sessão concluída. O reinício de uma OTA bem-sucedida começa uma nova sequência de sessões.

8. Manifesto `version.json`

```
{
  "version": "2.0",
  "url":
    "https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/firmware_v2.bin"
}
```

9. Passos para executar

1. Abrir o repositório indicado e conferir `version.json` e `firmware_v2.bin` na raiz da branch `main`. Os endereços diretos são públicos e não exigem login.
2. Abrir o projeto principal pelo link da primeira página. O circuito, a biblioteca e a tabela de partições já estão configurados. O código inicial é o firmware 1.0.
3. Iniciar a simulação em FW 1.0, observar o LED azul e aguardar três sessões completas. Registrar os horários impressos no Serial Monitor a 115200 baud.
4. Registrar consulta, download, sucesso da OTA e reinício automático. Confirmar FW 2.0, ordenação, mediana, histerese e mudança do LED sem substituir manualmente o código.
5. Consultar os logs e as capturas nos anexos deste relatório. Manter o repositório e o projeto públicos por pelo menos 10 dias após a entrega.

10. Compilação e bibliotecas

As duas versões usam o core Arduino ESP32 2.0.17, placa `esp32:esp32:esp32` e `ArduinoJson` 7.4.2. O arquivo `compile.sh` refaz os binários. As partições `app0` e `app1` têm 0x140000 bytes cada. O arquivo usado na OTA contém apenas a aplicação 2.0.

WiFi conecta à rede; `WiFiClientSecure` fornece HTTPS; `HTTPClient` consulta o manifesto; `ArduinoJson` interpreta o JSON; `HTTPUpdate` baixa e grava o firmware. A tarefa `FreeRTOS` já está incluída no core. Neste laboratório, `setInsecure()` não valida o certificado HTTPS; em produção, seriam necessárias validação do servidor e autenticidade do firmware.

Testes e tratamento de erros

11. Verificações realizadas

A lógica foi testada localmente e a aplicação foi executada no Wokwi. A demonstração principal verifica a coleta e a atualização remota. Uma cópia separada recebe valores fixos para verificar os limites da histerese.

Teste	Resultado observado
1. Firmware 1.0	Passou no Wokwi: cinco leituras por sessão, valores entre 10 e 20 cm, média e LED azul.
2. Sessões de 48 s	Passou no Wokwi: inícios em 0/48000/96000 ms e leituras a cada 2 s. O retorno de millis() a zero foi testado localmente.
3. Identificar versão nova	Passou no Wokwi: após a leitura em 104000 ms, consultou o GitHub e identificou a versão 2.0.
4. OTA e reinício em 2.0	Gravação OTA e reinício automático em 2.0 observados. Ver log da execução principal no anexo.
5. Média e mediana	Passou: média, vetor original, cópia ordenada e mediana exibidos após o reinício. Cálculos conferidos nos logs.
6. Mediana >= 16	Passou no Wokwi: mediana 16 ativa ALERTA e LED vermelho.
7. Faixa intermediária	Passou no teste controlado: mediana 15 mantém NORMAL e também mantém ALERTA, conforme o estado anterior.
8. Mediana <= 14	Passou no teste controlado: mediana 14 retorna a NORMAL; LED verde verificado.

12. Tratamento de erros verificado

Sem Wi-Fi, a tentativa termina após 15 segundos com mensagem. Falhas de acesso ao manifesto mostram o código HTTP ou erro de conexão. JSON inválido é rejeitado. Versão igual, inferior ou inválida não dispara download. Falhas de download ou de gravação mostram o código e a descrição retornados por HTTPUpdate. O programa não anuncia sucesso quando a atualização falha.

Os cinco cenários foram executados no projeto separado de erros. Foram observados: falha de Wi-Fi, HTTP 404 para o manifesto, versão igual sem download, erro -102 para binário ausente e erro -106 para cabeçalho inválido. O programa continuou medindo após as falhas.

Arquivos e evidências locais

13. Binário entregue

O arquivo `firmware_v2.bin` está incluído no pacote do projeto e como anexo interno deste PDF. Ele foi compilado a partir do código 2.0 apresentado no apêndice. O hash abaixo permite conferir se o arquivo publicado corresponde ao binário entregue.

Tamanho: 942.304 bytes.

SHA-256:

```
6e688739974a8424185950dda293e457e3b8d68246255a426e5382023ac88cb
```

Para extrair o anexo, use um leitor de PDF com painel de anexos, como Adobe Acrobat Reader. O arquivo no repositório deve ter o mesmo hash. Os fontes completos, `version.json` e `diagram.json` também estão anexados.

14. Verificações locais complementares

Os testes locais passaram para média, ordenação, mediana, histerese, temporização, retorno de `millis()` a zero e início da consulta após três sessões. A conferência automática dos logs reais também passou para 13 sessões completas. Os registros estão na pasta `evidencias`.

15. Compilação

Firmware 1.0: compilado para ESP32.

Sketch uses 935409 bytes (71%) of program storage space. Maximum is 1310720 bytes.

Firmware 2.0: compilado para ESP32.

Sketch uses 935729 bytes (71%) of program storage space. Maximum is 1310720 bytes.

Em consultas posteriores ao reboot, também ocorreram falhas de conexão e acesso ao manifesto. Elas foram informadas no Serial e a coleta continuou. Isso não impediu a OTA já concluída. Os avisos de core dump na inicialização indicam que não foi reservada uma partição para diagnóstico de falhas; não correspondem a falha na atualização.

16. Conclusão

A versão 1.0 realiza a coleta periódica e calcula a média. A versão 2.0 acrescenta a mediana e usa dois limites para evitar mudanças de estado perto de um único valor. A OTA permite levar essa evolução ao dispositivo instalado em campo. A execução no Wokwi comprovou a atualização remota e os estados do sistema. Os testes de erro também permitiram corrigir a reconexão Wi-Fi: uma tentativa anterior é encerrada antes de iniciar outra.

Referências

[Wokwi: ESP32](#)

[Wokwi: Wi-Fi](#)

[Espressif: partições](#)

[Espressif: OTA](#)

Enunciado: MORGS - CP2, professor Marcelo Fernando Morgantini.

Apêndice A - Firmware 1.0

Código completo: firmware_v1/firmware_v1.ino | linhas 1 a 56

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <HTTPUpdate.h>
#include <ArduinoJson.h>
#include <esp_system.h>

const char* VERSAO = "1.0";
// Configure com a URL RAW do version.json do seu repositório público.
const char* MANIFESTO_URL =
    "https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/version.json";
const char* REDE = "Wokwi-GUEST";
const int LED_R = 25, LED_G = 26, LED_B = 27;
const int QUANTIDADE = 5;
const uint32_t INTERVALO_LEITURA = 2000;
const uint32_t INTERVALO_SESSAO = 48000;
int leituras[QUANTIDADE];
int indice = 0;
int sessoesConcluidas = 0;
uint32_t inicioSessao;
uint32_t primeiraSessao;
bool alerta = false;
// A rede roda em uma tarefa para não atrasar as leituras do loop.
volatile bool otaEmAndamento = false;
int ultimaSessaoConsultada = 0;

void definirLed(bool vermelho, bool verde, bool azul) {
    digitalWrite(LED_R, vermelho);
    digitalWrite(LED_G, verde);
    digitalWrite(LED_B, azul);
}

float calcularMedia(const int valores[]) {
    int soma = 0;
    for (int i = 0; i < QUANTIDADE; i++) soma += valores[i];
    return soma / float(QUANTIDADE);
}

void concluirSessao() {
    Serial.printf("Media da sessao: %.1f cm\n", calcularMedia(leituras));

    sessoesConcluidas++;
    Serial.println("Sessao concluida. Proximo inicio: 48 s apos o anterior.");
}

void anunciarSessao() {
    Serial.printf("\nSessao %d | inicio relativo: %lu ms\n",
        sessoesConcluidas + 1,
        (unsigned long)(inicioSessao - primeiraSessao));
}

void atualizarLeituras() {
    uint32_t agora = millis();
    if (indice == QUANTIDADE &&
```

Apêndice A - Firmware 1.0

Código completo: firmware_v1/firmware_v1.ino | linhas 57 a 112

```
uint32_t( agora - inicioSessao) >= INTERVALO_SESSAO) {
    inicioSessao += INTERVALO_SESSAO;
    indice = 0;
    anunciarSessao();
}
if (indice < QUANTIDADE &&
    uint32_t( agora - inicioSessao) >= indice * INTERVALO_LEITURA) {
    leituras[indice] = random(10, 21); // 10 a 20, incluindo os extremos.
    Serial.printf("Leitura %d: %d cm | t=%lu ms\n", indice + 1,
        leituras[indice], (unsigned long)( agora - primeiraSessao));
    indice++;
    if (indice == QUANTIDADE) concluirSessao();
}
}

bool versaoMaisNova(const char* remota, const char* local) {
    int rMaior, rMenor, lMaior, lMenor;
    char sobra;
    // O projeto usa o formato numerico MAIOR.MENOR.
    if (sscanf(remota, "%d.%d%c", &rMaior, &rMenor, &sobra) != 2 ||
        sscanf(local, "%d.%d%c", &lMaior, &lMenor, &sobra) != 2 ||
        rMaior < 0 || rMenor < 0 || lMaior < 0 || lMenor < 0) {
        Serial.println("Erro: formato de versao invalido.");
        return false;
    }
    return rMaior > lMaior || (rMaior == lMaior && rMenor > lMenor);
}

bool conectarWiFi() {
    if (WiFi.status() == WL_CONNECTED) return true;
    Serial.printf("Conectando a rede %s...\n", REDE);
    WiFi.disconnect(); // Encerra uma tentativa anterior que ainda esteja ativa.
    delay(100);
    WiFi.mode(WIFI_STA);
    WiFi.begin(REDE, "", 6);
    uint32_t inicio = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - inicio < 15000) {
        delay(100); // Somente a tarefa de rede espera; o loop segue medindo.
    }
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("Erro: sem conexao Wi-Fi. Tentaremos na proxima sessao.");
        return false;
    }
    Serial.print("Wi-Fi conectado. IP: ");
    Serial.println(WiFi.localIP());
    return true;
}

void consultarAtualizacao() {
    if (String(MANIFESTO_URL).indexOf("SEU_USUARIO") >= 0) {
        Serial.println("OTA pendente: configure MANIFESTO_URL antes de publicar.");
        return;
    }
    if (!conectarWiFi()) return;
    WiFiClientSecure cliente;
    // Simplificacao para este laboratorio, sem dados privados.
```

Apêndice A - Firmware 1.0

Código completo: `firmware_v1/firmware_v1.ino` | linhas 113 a 168

```
// Em producao, validar o certificado do servidor com setCACert().
cliente.setInsecure();
HTTPClient http;
http.setConnectTimeout(10000);
http.setTimeout(15000);
http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
Serial.print("Consultando manifesto: ");
Serial.println(MANIFESTO_URL);
if (!http.begin(cliente, MANIFESTO_URL)) {
    Serial.println("Erro: nao foi possivel iniciar a consulta do manifesto.");
    return;
}
int codigo = http.GET();
if (codigo != HTTP_CODE_OK) {
    Serial.printf("Erro: manifesto inacessivel. HTTP/erro: %d\n", codigo);
    http.end();
    return;
}
JsonDocument documento;
DeserializationError erro = deserializeJson(documento, http.getString());
http.end();
if (erro || !documento["version"].is<const char*>() ||
    !documento["url"].is<const char*>()) {
    Serial.println("Erro: manifesto invalido (version e url obrigatorios).");
    return;
}
String versaoRemota = documento["version"].as<String>();
String url = documento["url"].as<String>();
Serial.printf("Versao instalada: %s | disponivel: %s\n",
    VERSAO, versaoRemota.c_str());
if (!versaoMaisNova(versaoRemota.c_str(), VERSAO)) {
    Serial.println("Sem atualizacao: versao atual ou remota nao superior/valida.");
    return;
}
if (!url.startsWith("https://")) {
    Serial.println("Erro: use uma URL HTTPS direta para o firmware.");
    return;
}
Serial.print("Atualizacao disponivel. Baixando: ");
Serial.println(url);
WiFiClientSecure download;
download.setInsecure();
HTTPUpdate atualizador;
atualizador.rebootOnUpdate(false);
atualizador.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
t_httpUpdate_return resultado = atualizador.update(download, url);
if (resultado == HTTP_UPDATE_OK) {
    Serial.println("OTA gravada com sucesso. Reiniciando o ESP32...");
    Serial.flush();
    ESP.restart();
} else if (resultado == HTTP_UPDATE_NO_UPDATES) {
    Serial.println("Servidor nao enviou um novo firmware.");
} else {
    Serial.printf("Erro no download/gravação OTA: %d - %s\n",
        atualizador.getLastErrorCode(),
        atualizador.getLastErrorMessage().c_str());
}
```

Apêndice A - Firmware 1.0

Código completo: firmware_v1/firmware_v1.ino | linhas 169 a 210

```
        Serial.println("Firmware atual mantido. Nova tentativa na proxima sessao.");
    }
}

void tarefaOTA(void*) {
    consultarAtualizacao();
    otaEmAndamento = false;
    vTaskDelete(nullptr);
}

void verificarOTA() {
    // A primeira tentativa so ocorre depois de tres sessoes completas.
    if (sessoesConcluidas >= 3 && !otaEmAndamento &&
        ultimaSessaoConsultada != sessoesConcluidas) {
        ultimaSessaoConsultada = sessoesConcluidas;
        otaEmAndamento = true;
        if (xTaskCreate(tarefaOTA, "OTA", 12288, nullptr, 1, nullptr) != pdPASS) {
            otaEmAndamento = false;
            Serial.println("Erro: nao foi possivel iniciar a tarefa OTA.");
        }
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(LED_R, OUTPUT);
    pinMode(LED_G, OUTPUT);
    pinMode(LED_B, OUTPUT);
    definirLed(false, false, true);
    randomSeed(esp_random());
    Serial.println("=====");
    Serial.printf("MONITORAMENTO DE VEGETACAO - FW %s\n", VERSAO);
    Serial.println("=====");
    primeiraSessao = inicioSessao = millis();
    anunciarSessao();
}

void loop() {
    atualizarLeituras();
    verificarOTA();
    delay(1); // Cede tempo ao sistema, sem uma espera longa entre leituras.
}
```

Apêndice B - Firmware 2.0

Código completo: `firmware_v2/firmware_v2.ino` | linhas 1 a 56

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <HTTPUpdate.h>
#include <ArduinoJson.h>
#include <esp_system.h>

const char* VERSAO = "2.0";
// Configure com a URL RAW do version.json do seu repositório público.
const char* MANIFESTO_URL =
    "https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/version.json";
const char* REDE = "Wokwi-GUEST";
const int LED_R = 25, LED_G = 26, LED_B = 27;
const int QUANTIDADE = 5;
const uint32_t INTERVALO_LEITURA = 2000;
const uint32_t INTERVALO_SESSAO = 48000;
int leituras[QUANTIDADE];
int indice = 0;
int sessoesConcluidas = 0;
uint32_t inicioSessao;
uint32_t primeiraSessao;
bool alerta = false;
// A rede roda em uma tarefa para não atrasar as leituras do loop.
volatile bool otaEmAndamento = false;
int ultimaSessaoConsultada = 0;

void definirLed(bool vermelho, bool verde, bool azul) {
    digitalWrite(LED_R, vermelho);
    digitalWrite(LED_G, verde);
    digitalWrite(LED_B, azul);
}

float calcularMedia(const int valores[]) {
    int soma = 0;
    for (int i = 0; i < QUANTIDADE; i++) soma += valores[i];
    return soma / float(QUANTIDADE);
}

void ordenarCopia(const int original[], int ordenado[]) {
    for (int i = 0; i < QUANTIDADE; i++) ordenado[i] = original[i];
    for (int i = 0; i < QUANTIDADE - 1; i++) {
        for (int j = 0; j < QUANTIDADE - 1 - i; j++) {
            if (ordenado[j] > ordenado[j + 1]) {
                int aux = ordenado[j];
                ordenado[j] = ordenado[j + 1];
                ordenado[j + 1] = aux;
            }
        }
    }
}

bool aplicarHisterese(int mediana, bool estadoAnterior) {
    if (mediana >= 16) return true;
    if (mediana <= 14) return false;
    return estadoAnterior;
}
```

Apêndice B - Firmware 2.0

Código completo: firmware_v2/firmware_v2.ino | linhas 57 a 112

```
}

void mostrarVetor(const char* titulo, const int valores[]) {
    Serial.print(titulo);
    for (int i = 0; i < QUANTIDADE; i++) Serial.printf("%d ", valores[i]);
    Serial.println();
}

void concluirSessao() {
    Serial.printf("Media da sessao: %.1f cm\n", calcularMedia(leituras));
    int ordenado[QUANTIDADE];
    ordenarCopia(leituras, ordenado);
    mostrarVetor("Ordem original: ", leituras);
    mostrarVetor("Ordem crescente: ", ordenado);
    int mediana = ordenado[2];
    alerta = aplicarHisterese(mediana, alerta);
    definirLed(alerta, !alerta, false);
    Serial.printf("Mediana: %d cm | Estado: %s\n", mediana,
        alerta ? "ALERTA" : "NORMAL");
    sessoesConcluidas++;
    Serial.println("Sessao concluida. Proximo inicio: 48 s apos o anterior.");
}

void anunciarSessao() {
    Serial.printf("\nSessao %d | inicio relativo: %lu ms\n",
        sessoesConcluidas + 1,
        (unsigned long)(inicioSessao - primeiraSessao));
}

void atualizarLeituras() {
    uint32_t agora = millis();
    if (indice == QUANTIDADE &&
        uint32_t(agora - inicioSessao) >= INTERVALO_SESSAO) {
        inicioSessao += INTERVALO_SESSAO;
        indice = 0;
        anunciarSessao();
    }
    if (indice < QUANTIDADE &&
        uint32_t(agora - inicioSessao) >= indice * INTERVALO_LEITURA) {
        leituras[indice] = random(10, 21); // 10 a 20, incluindo os extremos.
        Serial.printf("Leitura %d: %d cm | t=%lu ms\n", indice + 1,
            leituras[indice], (unsigned long)(agora - primeiraSessao));
        indice++;
        if (indice == QUANTIDADE) concluirSessao();
    }
}

bool versaoMaisNova(const char* remota, const char* local) {
    int rMaior, rMenor, lMaior, lMenor;
    char sobra;
    // O projeto usa o formato numerico MAIOR.MENOR.
    if (sscanf(remota, "%d.%d%c", &rMaior, &rMenor, &sobra) != 2 ||
        sscanf(local, "%d.%d%c", &lMaior, &lMenor, &sobra) != 2 ||
        rMaior < 0 || rMenor < 0 || lMaior < 0 || lMenor < 0) {
        Serial.println("Erro: formato de versao invalido.");
        return false;
    }
}
```

Apêndice B - Firmware 2.0

Código completo: `firmware_v2/firmware_v2.ino` | linhas 113 a 168

```
}
    return rMaior > lMaior || (rMaior == lMaior && rMenor > lMenor);
}

bool conectarWiFi() {
    if (WiFi.status() == WL_CONNECTED) return true;
    Serial.printf("Conectando a rede %s...\n", REDE);
    WiFi.disconnect(); // Encerra uma tentativa anterior que ainda esteja ativa.
    delay(100);
    WiFi.mode(WIFI_STA);
    WiFi.begin(REDE, "", 6);
    uint32_t inicio = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - inicio < 15000) {
        delay(100); // Somente a tarefa de rede espera; o loop segue medindo.
    }
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("Erro: sem conexao Wi-Fi. Tentaremos na proxima sessao.");
        return false;
    }
    Serial.print("Wi-Fi conectado. IP: ");
    Serial.println(WiFi.localIP());
    return true;
}

void consultarAtualizacao() {
    if (String(MANIFESTO_URL).indexOf("SEU_USUARIO") >= 0) {
        Serial.println("OTA pendente: configure MANIFESTO_URL antes de publicar.");
        return;
    }
    if (!conectarWiFi()) return;
    WiFiClientSecure cliente;
    // Simplificacao para este laboratorio, sem dados privados.
    // Em producao, validar o certificado do servidor com setCACert().
    cliente.setInsecure();
    HTTPClient http;
    http.setConnectTimeout(10000);
    http.setTimeout(15000);
    http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
    Serial.print("Consultando manifesto: ");
    Serial.println(MANIFESTO_URL);
    if (!http.begin(cliente, MANIFESTO_URL)) {
        Serial.println("Erro: nao foi possivel iniciar a consulta do manifesto.");
        return;
    }
    int codigo = http.GET();
    if (codigo != HTTP_CODE_OK) {
        Serial.printf("Erro: manifesto inacessivel. HTTP/erro: %d\n", codigo);
        http.end();
        return;
    }
    JsonDocument documento;
    DeserializationError erro = deserializeJson(documento, http.getString());
    http.end();
    if (erro || !documento["version"].is<const char*>() ||
        !documento["url"].is<const char*>()) {
        Serial.println("Erro: manifesto invalido (version e url obrigatorios).");
    }
}
```

Apêndice B - Firmware 2.0

Código completo: firmware_v2/firmware_v2.ino | linhas 169 a 224

```
        return;
    }
    String versaoRemota = documento["version"].as<String>();
    String url = documento["url"].as<String>();
    Serial.printf("Versao instalada: %s | disponivel: %s\n",
        VERSAO, versaoRemota.c_str());
    if (!versaoMaisNova(versaoRemota.c_str(), VERSAO)) {
        Serial.println("Sem atualizacao: versao atual ou remota nao superior/valida.");
        return;
    }
    if (!url.startsWith("https://")) {
        Serial.println("Erro: use uma URL HTTPS direta para o firmware.");
        return;
    }
    Serial.print("Atualizacao disponivel. Baixando: ");
    Serial.println(url);
    WiFiClientSecure download;
    download.setInsecure();
    HTTPUpdate atualizador;
    atualizador.rebootOnUpdate(false);
    atualizador.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
    t_httpUpdate_return resultado = atualizador.update(download, url);
    if (resultado == HTTP_UPDATE_OK) {
        Serial.println("OTA gravada com sucesso. Reiniciando o ESP32...");
        Serial.flush();
        ESP.restart();
    } else if (resultado == HTTP_UPDATE_NO_UPDATES) {
        Serial.println("Servidor nao enviou um novo firmware.");
    } else {
        Serial.printf("Erro no download/gravação OTA: %d - %s\n",
            atualizador.getLastErrorCode(),
            atualizador.getLastErrorMessage().c_str());
        Serial.println("Firmware atual mantido. Nova tentativa na proxima sessao.");
    }
}

void tarefaOTA(void*) {
    consultarAtualizacao();
    otaEmAndamento = false;
    vTaskDelete(nullptr);
}

void verificarOTA() {
    // A primeira tentativa so ocorre depois de tres sessoes completas.
    if (sessoesConcluidas >= 3 && !otaEmAndamento &&
        ultimaSessaoConsultada != sessoesConcluidas) {
        ultimaSessaoConsultada = sessoesConcluidas;
        otaEmAndamento = true;
        if (xTaskCreate(tarefaOTA, "OTA", 12288, nullptr, 1, nullptr) != pdPASS) {
            otaEmAndamento = false;
            Serial.println("Erro: nao foi possivel iniciar a tarefa OTA.");
        }
    }
}

void setup() {
```

Apêndice B - Firmware 2.0

Código completo: firmware_v2/firmware_v2.ino | linhas 225 a 242

```
Serial.begin(115200);
pinMode(LED_R, OUTPUT);
pinMode(LED_G, OUTPUT);
pinMode(LED_B, OUTPUT);
definirLed(false, true, false);
randomSeed(esp_random());
Serial.println("=====");
Serial.printf("MONITORAMENTO DE VEGETACAO - FW %s\n", VERSAO);
Serial.println("=====");
primeiraSessao = inicioSessao = millis();
anunciarSessao();
}

void loop() {
    atualizarLeituras();
    verificarOTA();
    delay(1); // Cede tempo ao sistema, sem uma espera longa entre leituras.
}
```

Apêndice C - Configuração

partitions.csv

```
# Name, Type, SubType, Offset, Size
nvs, data, nvs, 0x9000, 0x5000
otadata, data, ota, 0xe000, 0x2000
app0, app, ota_0, 0x10000, 0x140000
app1, app, ota_1, 0x150000, 0x140000
spiffs, data, spiffs, 0x290000, 0x170000
```

libraries.txt

ArduinoJson@7.4.2

Organização dos arquivos

```
Motiva-CP2-OTA/
  version.json
  firmware_v1.bin
  firmware_v2.bin
  firmware_v1/firmware_v1.ino
  firmware_v2/firmware_v2.ino
  wokwi/sketch.ino
  wokwi/diagram.json
  wokwi/libraries.txt
  wokwi/partitions.csv
  README.md
  PUBLICAR-E-TESTAR.md
  evidencias/
  tests/testar.py
  output/pdf/Relatorio-Motiva-CP2.pdf
```

Como repetir a demonstração

Abrir o projeto principal no Wokwi e iniciar a simulação. Aguardar três sessões completas e o download do binário. O tempo de parede pode ser maior que o tempo simulado. Conferir a mensagem de sucesso e o cabeçalho FW 2.0 após o reinício. Usar a cópia de histerese para reproduzir os limites sem depender dos valores aleatórios.

Anexo - OTA e funcionamento

Saída da aplicação. Log original completo disponível nos anexos internos do PDF.

Fonte: <https://wokwi.com/projects/475988461020262401>
Execucao final, firmware corrigido, 23/09/2026.

```
=====
MONITORAMENTO DE VEGETACAO - FW 1.0
=====

Sessao 1 | inicio relativo: 0 ms
Leitura 1: 20 cm | t=10 ms
Leitura 2: 18 cm | t=2000 ms
Leitura 3: 18 cm | t=4000 ms
Leitura 4: 11 cm | t=6000 ms
Leitura 5: 19 cm | t=8000 ms
Media da sessao: 17.2 cm
Sessao concluida. Proximo inicio: 48 s apos o anterior.

Sessao 2 | inicio relativo: 48000 ms
Leitura 1: 18 cm | t=48000 ms
Leitura 2: 14 cm | t=50000 ms
Leitura 3: 20 cm | t=52000 ms
Leitura 4: 11 cm | t=54000 ms
Leitura 5: 18 cm | t=56000 ms
Media da sessao: 16.2 cm
Sessao concluida. Proximo inicio: 48 s apos o anterior.

Sessao 3 | inicio relativo: 96000 ms
Leitura 1: 12 cm | t=96000 ms
Leitura 2: 14 cm | t=98000 ms
Leitura 3: 19 cm | t=100000 ms
Leitura 4: 14 cm | t=102000 ms
Leitura 5: 15 cm | t=104000 ms
Media da sessao: 14.8 cm
Sessao concluida. Proximo inicio: 48 s apos o anterior.
Conectando a rede Wokwi-GUEST...
Wi-Fi conectado. IP: 10.10.0.2
Consultando manifesto: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/version.json
Versao instalada: 1.0 | disponivel: 2.0
Atualizacao disponivel. Baixando: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/firmware_v2.bin

Sessao 4 | inicio relativo: 144000 ms
Leitura 1: 16 cm | t=144000 ms
Leitura 2: 15 cm | t=146000 ms
Leitura 3: 12 cm | t=148000 ms
Leitura 4: 17 cm | t=150000 ms
Leitura 5: 13 cm | t=152014 ms
Media da sessao: 14.6 cm
Sessao concluida. Proximo inicio: 48 s apos o anterior.

Sessao 5 | inicio relativo: 192000 ms
Leitura 1: 19 cm | t=192001 ms
OTA gravada com sucesso. Reiniciando o ESP32...

rst:0xc (SW_CPU_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
=====
MONITORAMENTO DE VEGETACAO - FW 2.0
=====

Sessao 1 | inicio relativo: 0 ms
Leitura 1: 16 cm | t=11 ms
Leitura 2: 17 cm | t=2000 ms
Leitura 3: 19 cm | t=4000 ms
Leitura 4: 11 cm | t=6000 ms
```

Anexo - OTA e funcionamento

Saída da aplicação. Log original completo disponível nos anexos internos do PDF.

```
Leitura 5: 11 cm | t=8000 ms
Media da sessao: 14.8 cm
Ordem original: 16 17 19 11 11
Ordem crescente: 11 11 16 17 19
Mediana: 16 cm | Estado: ALERTA
Sessao concluida. Proximo inicio: 48 s apos o anterior.
```

```
Sessao 2 | inicio relativo: 48000 ms
Leitura 1: 11 cm | t=48000 ms
Leitura 2: 19 cm | t=50000 ms
Leitura 3: 10 cm | t=52000 ms
Leitura 4: 17 cm | t=54000 ms
Leitura 5: 14 cm | t=56000 ms
Media da sessao: 14.2 cm
Ordem original: 11 19 10 17 14
Ordem crescente: 10 11 14 17 19
Mediana: 14 cm | Estado: NORMAL
Sessao concluida. Proximo inicio: 48 s apos o anterior.
```

```
Sessao 3 | inicio relativo: 96000 ms
Leitura 1: 16 cm | t=96000 ms
Leitura 2: 13 cm | t=98000 ms
Leitura 3: 18 cm | t=100000 ms
Leitura 4: 20 cm | t=102000 ms
Leitura 5: 13 cm | t=104000 ms
Media da sessao: 16.0 cm
Ordem original: 16 13 18 20 13
Ordem crescente: 13 13 16 18 20
Mediana: 16 cm | Estado: ALERTA
Sessao concluida. Proximo inicio: 48 s apos o anterior.
Conectando a rede Wokwi-GUEST...
Erro: sem conexao Wi-Fi. Tentaremos na proxima sessao.
```

```
Sessao 4 | inicio relativo: 144000 ms
Leitura 1: 16 cm | t=144000 ms
Leitura 2: 15 cm | t=146000 ms
Leitura 3: 19 cm | t=148000 ms
Leitura 4: 13 cm | t=150000 ms
Leitura 5: 13 cm | t=152000 ms
Media da sessao: 15.2 cm
Ordem original: 16 15 19 13 13
Ordem crescente: 13 13 15 16 19
Mediana: 15 cm | Estado: ALERTA
Sessao concluida. Proximo inicio: 48 s apos o anterior.
Consultando manifesto: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/version.json
Erro: manifesto inacessivel. HTTP/erro: -1
```

Anexo - Histerese

Saída da aplicação. Log original completo disponível nos anexos internos do PDF.

Fonte: <https://wokwi.com/projects/475988911733956609>

Dados fixos, sessões de 12 s apenas para testar histerese.

DIAGNOSTICO HISTERese: dados fixos; nao comprova OTA nem periodo de 48 s.

=====

MONITORAMENTO DE VEGETACAO - FW 2.0

=====

Sessao 1 | inicio relativo: 0 ms

Leitura 1: 15 cm | t=0 ms

Leitura 2: 14 cm | t=2000 ms

Leitura 3: 15 cm | t=4000 ms

Leitura 4: 16 cm | t=6000 ms

Leitura 5: 15 cm | t=8000 ms

Media da sessao: 15.0 cm

Ordem original: 15 14 15 16 15

Ordem crescente: 14 15 15 15 16

Mediana: 15 cm | Estado: NORMAL

Sessao concluida. DIAGNOSTICO: proximo inicio em 12 s apos o anterior.

Sessao 2 | inicio relativo: 12000 ms

Leitura 1: 18 cm | t=12000 ms

Leitura 2: 16 cm | t=14000 ms

Leitura 3: 15 cm | t=16000 ms

Leitura 4: 16 cm | t=18000 ms

Leitura 5: 20 cm | t=20000 ms

Media da sessao: 17.0 cm

Ordem original: 18 16 15 16 20

Ordem crescente: 15 16 16 18 20

Mediana: 16 cm | Estado: ALERTA

Sessao concluida. DIAGNOSTICO: proximo inicio em 12 s apos o anterior.

Sessao 3 | inicio relativo: 24000 ms

Leitura 1: 15 cm | t=24000 ms

Leitura 2: 14 cm | t=26000 ms

Leitura 3: 15 cm | t=28000 ms

Leitura 4: 16 cm | t=30000 ms

Leitura 5: 15 cm | t=32000 ms

Media da sessao: 15.0 cm

Ordem original: 15 14 15 16 15

Ordem crescente: 14 15 15 15 16

Mediana: 15 cm | Estado: ALERTA

Sessao concluida. DIAGNOSTICO: proximo inicio em 12 s apos o anterior.

Sessao 4 | inicio relativo: 36000 ms

Leitura 1: 14 cm | t=36000 ms

Leitura 2: 10 cm | t=38000 ms

Leitura 3: 20 cm | t=40000 ms

Leitura 4: 12 cm | t=42000 ms

Leitura 5: 14 cm | t=44000 ms

Media da sessao: 14.0 cm

Ordem original: 14 10 20 12 14

Ordem crescente: 10 12 14 14 20

Mediana: 14 cm | Estado: NORMAL

Sessao concluida. DIAGNOSTICO: proximo inicio em 12 s apos o anterior.

Anexo - Tratamento de erros

Saída da aplicação. Log original completo disponível nos anexos internos do PDF.

Diagnostico de erros executado no Wokwi em 23/09/2026.
Projeto: <https://wokwi.com/projects/475989558168554497>

```
=====
MONITORAMENTO DE VEGETACAO - FW 1.0
=====

Sessao 1 | inicio relativo: 0 ms
DIAGNOSTICO: erros de rede e OTA, sem substituir o teste principal.
TESTE 1: SEM WI-FI
Conectando a rede REDE-INEXISTENTE-CP2...
Leitura 1: 15 cm | t=11 ms
Leitura 2: 10 cm | t=2000 ms
Leitura 3: 12 cm | t=4000 ms
Leitura 4: 14 cm | t=6000 ms
Leitura 5: 12 cm | t=8000 ms
Media da sessao: 12.6 cm
Sessao concluida. Proximo inicio: 48 s apos o anterior.
Erro: sem conexao Wi-Fi. Tentaremos na proxima sessao.

TESTE 2: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/inexistente.json
Conectando a rede Wokwi-GUEST...
Wi-Fi conectado. IP: 10.10.0.2
Consultando manifesto: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/inexistente.json
Erro: manifesto inacessivel. HTTP/erro: 404

TESTE 3: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/Atual.json
Consultando manifesto: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/Atual.json
Versao instalada: 1.0 | disponivel: 1.0
Sem atualizacao: versao atual ou remota nao superior/valida.

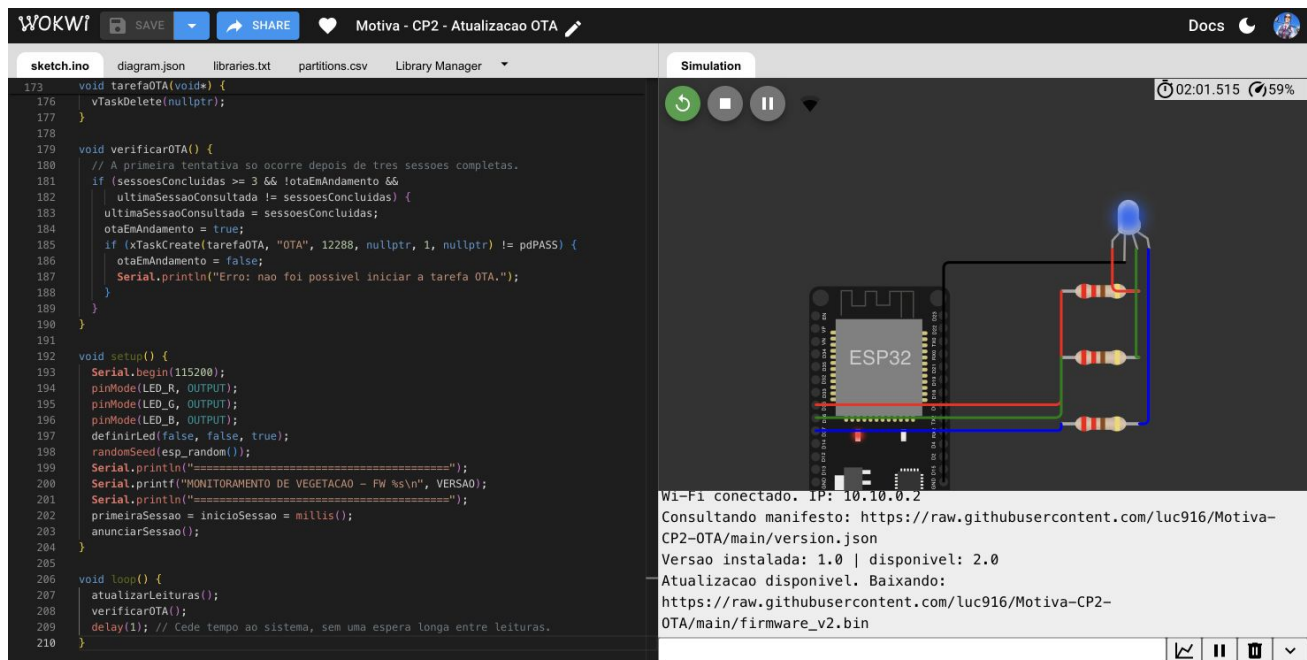
TESTE 4: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/bin-ausente.json
Consultando manifesto: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/bin-ausente.json
Versao instalada: 1.0 | disponivel: 2.0
Atualizacao disponivel. Baixando: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/arquivo-inexistente.bin
Erro no download/gravação OTA: -102 - File Not Found (404)
Firmware atual mantido. Nova tentativa na proxima sessao.

TESTE 5: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/bin-invalido.json
Consultando manifesto: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/bin-invalido.json
Versao instalada: 1.0 | disponivel: 2.0
Atualizacao disponivel. Baixando: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/tests/ota/invalido.bin

Sessao 2 | inicio relativo: 48000 ms
Leitura 1: 17 cm | t=48000 ms
Leitura 2: 17 cm | t=50000 ms
Erro no download/gravação OTA: -106 - Verify Bin Header Failed
Firmware atual mantido. Nova tentativa na proxima sessao.
DIAGNOSTICO CONCLUIDO: verificar cada mensagem acima.
Leitura 3: 19 cm | t=52000 ms
Leitura 4: 16 cm | t=54000 ms
Leitura 5: 16 cm | t=56000 ms
Media da sessao: 17.0 cm
Sessao concluida. Proximo inicio: 48 s apos o anterior.
```

Anexo - Demonstração no Wokwi

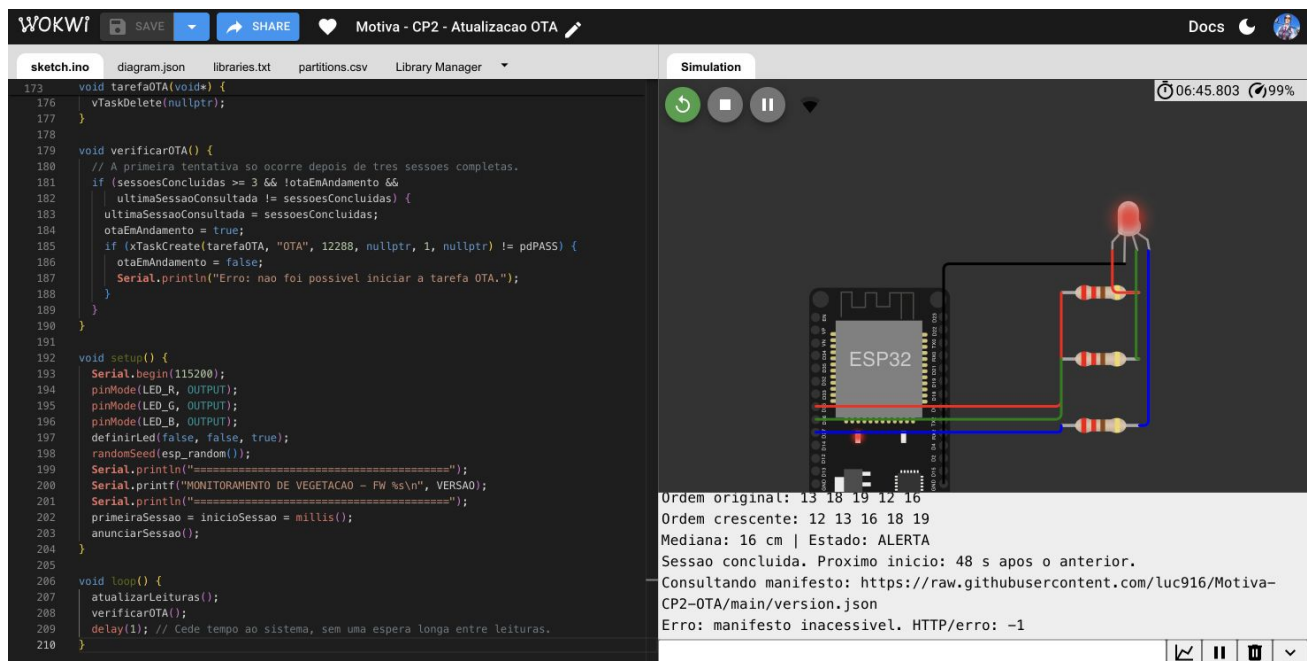
Firmware 1.0: LED azul e download remoto da versão 2.0.



The screenshot shows the Wokwi IDE interface. On the left, the sketch code is visible, showing functions for OTA updates and LED control. On the right, the simulation window displays an ESP32 board with a blue LED lit. The console output shows the following messages:

```
Wi-Fi conectado. IP: 10.10.0.2
Consultando manifesto: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/version.json
Versao instalada: 1.0 | disponivel: 2.0
Atualizacao disponivel. Baixando:
https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/firmware_v2.bin
```

Após OTA: ordenação, mediana e LED vermelho. A consulta posterior registrou falha de rede, tratada sem parar a coleta.



The screenshot shows the Wokwi IDE interface after the OTA update. The simulation window displays the ESP32 board with a red LED lit. The console output shows the following messages:

```
Ordem original: 13 18 19 12 16
Ordem crescente: 12 13 16 18 19
Mediana: 16 cm | Estado: ALERTA
Sessao concluida. Proximo inicio: 48 s apos o anterior.
Consultando manifesto: https://raw.githubusercontent.com/luc916/Motiva-CP2-OTA/main/version.json
Erro: manifesto inacessivel. HTTP/erro: -1
```

Anexo - Demonstração no Wokwi

Teste controlado: mediana de 15 cm mantém ALERTA e LED vermelho.

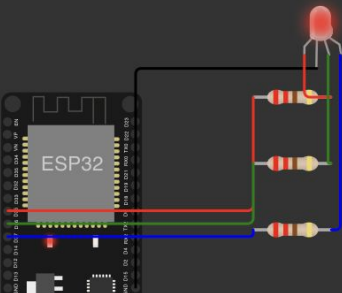
WOKWI SAVE SHARE Motiva CP2 - Teste de histerese (dados fixos) Docs

sketch.ino diagram.json libraries.txt partitions.csv Library Manager

```
206 void tarefaOTA(void*) {
207 }
208
209 void verificarOTA() {
210 // A primeira tentativa so ocorre depois de tres sessoes completas.
211 if (sessoesConcluidas >= 3 && !otaEmAndamento &&
212     ultimaSessaoConsultada != sessoesConcluidas) {
213     ultimaSessaoConsultada = sessoesConcluidas;
214     otaEmAndamento = true;
215     if (xTaskCreate(tarefaOTA, "OTA", 12288, nullptr, 1, nullptr) != pdPASS) {
216         otaEmAndamento = false;
217         Serial.println("Erro: nao foi possivel iniciar a tarefa OTA.");
218     }
219 }
220 }
221
222 void setup() {
223 Serial.begin(115200);
224 Serial.println("DIAGNOSTICO HISTERese: dados fixos; nao comprova OTA nem periodo de 48 s.");
225 pinMode(LED_R, OUTPUT);
226 pinMode(LED_G, OUTPUT);
227 pinMode(LED_B, OUTPUT);
228 digitalWrite(LED_R, true);
229 randomSeed(esp_random());
230 Serial.println("=====");
231 Serial.printf("MONITORAMENTO DE VEGETACAO - FW %s\n", VERSAO);
232 Serial.println("=====");
233 primeiraSessao = inicioSessao = millis();
234 anunciarSessao();
235 }
236
237 void loop() {
238 atualizarLeituras();
239 delay(1); // Cede tempo ao sistema, sem uma espera longa entre leituras.
240 }
```

Simulation 03:52.156 100%

Pause



Ordem crescente: 14 15 15 15 16
Mediana: 15 cm | Estado: ALERTA
Sessao concluida. DIAGNOSTICO: proximo inicio em 12 s apos o anterior.

Sessao 20 | inicio relativo: 228000 ms
Leitura 1: 14 cm | t=228000 ms
Leitura 2: 10 cm | t=230000 ms

Teste controlado: estado NORMAL e LED verde.

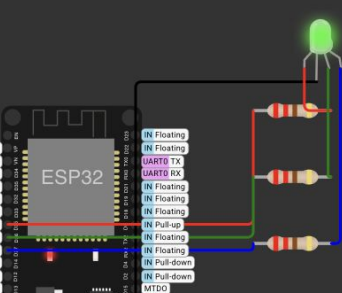
WOKWI SAVE SHARE Motiva CP2 - Teste de histerese (dados fixos) Docs

sketch.ino diagram.json libraries.txt partitions.csv Library Manager

```
206 void tarefaOTA(void*) {
207 }
208
209 void verificarOTA() {
210 // A primeira tentativa so ocorre depois de tres sessoes completas.
211 if (sessoesConcluidas >= 3 && !otaEmAndamento &&
212     ultimaSessaoConsultada != sessoesConcluidas) {
213     ultimaSessaoConsultada = sessoesConcluidas;
214     otaEmAndamento = true;
215     if (xTaskCreate(tarefaOTA, "OTA", 12288, nullptr, 1, nullptr) != pdPASS) {
216         otaEmAndamento = false;
217         Serial.println("Erro: nao foi possivel iniciar a tarefa OTA.");
218     }
219 }
220 }
221
222 void setup() {
223 Serial.begin(115200);
224 Serial.println("DIAGNOSTICO HISTERese: dados fixos; nao comprova OTA nem periodo de 48 s.");
225 pinMode(LED_R, OUTPUT);
226 pinMode(LED_G, OUTPUT);
227 pinMode(LED_B, OUTPUT);
228 digitalWrite(LED_R, false);
229 randomSeed(esp_random());
230 Serial.println("=====");
231 Serial.printf("MONITORAMENTO DE VEGETACAO - FW %s\n", VERSAO);
232 Serial.println("=====");
233 primeiraSessao = inicioSessao = millis();
234 anunciarSessao();
235 }
236
237 void loop() {
238 atualizarLeituras();
239 delay(1); // Cede tempo ao sistema, sem uma espera longa entre leituras.
240 }
```

Simulation 01:05.213 100%

Resume



Ordem crescente: 14 15 15 15 16
Mediana: 15 cm | Estado: NORMAL
Sessao concluida. DIAGNOSTICO: proximo inicio em 12 s apos o anterior.

Sessao 6 | inicio relativo: 60000 ms
Leitura 1: 18 cm | t=60000 ms
Leitura 2: 16 cm | t=62000 ms